GUIA PRÁTICO • 30 OUTUBRO 2026

Guia Oficial do Uso de Skills

Como instalar e utilizar as **11 skills do repositório**: qual chamar, o que informar, o que conferir e como seguir para a próxima etapa.



Onde consultar	Páginas
1. Visão Geral e Arquitetura	2
2. Inventário de Skills	3-4
3. Uso em um Projeto Real	5-9
A. Preparação	5
B. Feature	6-7
C. Deploy e Operação	8-9

Como ler os diagramas

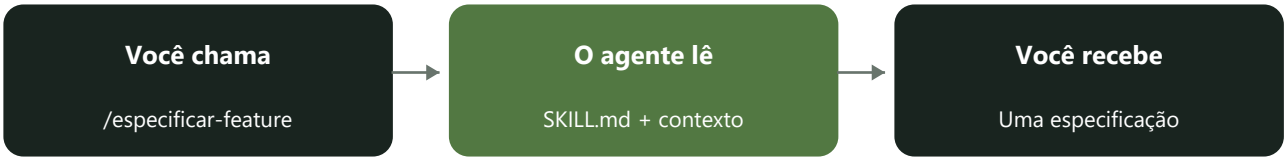
As setas mostram a sequência. Blocos tracejados indicam opções ou condições. Os comandos com barra funcionam onde o agente oferece esse suporte; também é possível pedir uma skill pelo nome.

Conteúdo: README.md e skills/*/SKILL.md da cópia local atual. O PDF AI-DLC fornecido foi referência apenas de design, não de conteúdo. Exemplos de chamados ilustram pedidos; não definem regras para o seu produto.

1. VISÃO GERAL E ARQUITETURA DE SKILLS

Como uma skill funciona

Uma skill é uma pasta com **SKILL.md**. O arquivo informa quando usar, quais passos executar, quais documentos consultar e quais limites respeitar. O procedimento é reutilizável; produto, stack e arquitetura pertencem ao projeto que o utiliza.



Arquivo	Uso pelas skills
AGENTS.md	Método de trabalho comum, instalado a partir do modelo.
CLAUDE.md	Importa @AGENTS.md no Claude Code.
HARNESS.md	Regras próprias do projeto; prevalecem sobre AGENTS.md em conflitos entre ambos.
PRD.md	Objetivo, escopo, histórias e regras de negócio.
TDD.md	Technical Design Document: stack, arquitetura, interfaces e configuração. Não significa Test-Driven Development aqui.
DESIGN.md	Estilo visual quando a tarefa envolve interface.
BDD.md e bdd/	Índice e cenários de comportamento. São convenções desta coleção.
README.md	Instalação, configuração e uso do produto.

COMO CHAMAR

/especificar-feature

Ou: “Use a skill especificar-feature para detalhar o encerramento de chamados.”

O README declara compatibilidade com Claude Code, Cursor, Copilot, Devin Desktop e agentes que aceitam SKILL.md. A seleção automática de uma skill depende do agente e da configuração; chamar pelo nome torna a intenção explícita.

O desenvolvimento orientado a especificação (SDD) faz parte do próprio repositório: especificação e plano aprovados precedem a implementação. As aprovações ocorrem nos pontos definidos por cada skill.

2. INVENTÁRIO DE SKILLS

Preparar e desenvolver

01 • iniciar-projeto

Objetivo: Criar instruções e estrutura documental inicial.
Peça: “Use iniciar-projeto na raiz e preserve os documentos existentes.”
Resultado e limite: Arquivos ausentes são criados; CLI do host, Context7 e Octocode são conferidos. Não define produto nem cria a aplicação.

02 • especificar-projeto

Objetivo: Especificar produto e arquitetura.
Peça: “Use especificar-projeto para um sistema de chamados; defina primeiro o PRD.”
Resultado e limite: PRD e TDD preenchidos; design somente com frontend; BDD e plano se escolhidos. Não implementa.

03 • implementar-sdd

Objetivo: Conduzir a funcionalidade de ponta a ponta.
Peça: “Use implementar-sdd para permitir encerrar chamados abertos.”
Resultado e limite: Especifica, planeja e implementa; oferece testes BDD, revisão e versionamento. Não atualiza docs nem publica release.

04 • especificar-feature

Objetivo: Detalhar uma funcionalidade antes do código.
Peça: “Use especificar-feature para definir regras, erros e cenários de encerramento.”
Resultado e limite: Spec no chat; gravação em arquivo e de cenários exige autorização. Planejamento usa o modo nativo do agente. Não implementa.

05 • criar-testes

Objetivo: Cobrir cenários BDD que ainda não possuem testes.
Peça: “Use criar-testes para os cenários pendentes de bdd/us-02.md.”
Resultado e limite: Testes rastreáveis por ID e relatório. Não muda produção, cenários nem testes existentes; @manual vira checklist.

06 • revisar-codigo

Objetivo: Analisar qualidade e sugerir melhorias.
Peça: “Use revisar-codigo no diff e nos testes novos.”
Resultado e limite: Relatório por criticidade sobre duplicação, SOLID, arquitetura e BDD. Não aplica correções nem refatorações.

Para uma etapa isolada, peça somente a skill correspondente. Para retomar implementar-sdd, indique a etapa já concluída; o fluxo não precisa refazer o trabalho existente.

2. INVENTÁRIO DE SKILLS

Versionar, publicar e ampliar

07 • versionar

Objetivo: Organizar commits, atualizar versão e enviar a branch.
Peça: "Use versionar para preparar commits por assunto, subir a versão e enviar a branch."
Resultado e limite: Plano e commits coesos, versão e envio quando autorizados. Pedido apenas de commit não autoriza versão ou push. Não cria tag.

08 • gerar-guardrails

Objetivo: Preparar CI e proteção da branch no GitHub.
Peça: "Use gerar-guardrails para exigir os testes no CI."
Resultado e limite: Workflows e proteção, ou indicação de pendência. Exige suíte existente; não faz commit nem push. Fora do GitHub, entrega plano.

09 • atualizar-docs

Objetivo: Sincronizar PRD, TDD e README com o código.
Peça: "Use atualizar-docs; ao final, não commite nem publique."
Resultado e limite: Documentos e marcador de revisão atualizados. Pergunta no início sobre commit, release ou nenhum. Não altera produção ou BDD.

10 • publicar-release

Objetivo: Criar e enviar a tag da versão já declarada.
Peça: "Use publicar-release para a versão do manifesto e acompanhe o pipeline."
Resultado e limite: Tag e estado da publicação. Exige árvore limpa; não commita nem eleva versão. Sem pipeline, não publica artefatos automaticamente.

11 • gerar-skill

Objetivo: Criar um procedimento reutilizável.
Peça: "Use gerar-skill para extrair um procedimento de revisão de contratos de API."
Resultado e limite: Rascunho aprovado e SKILL.md; oferece contribuição por PR ao catálogo. Não implementa o procedimento nem fixa decisões locais.

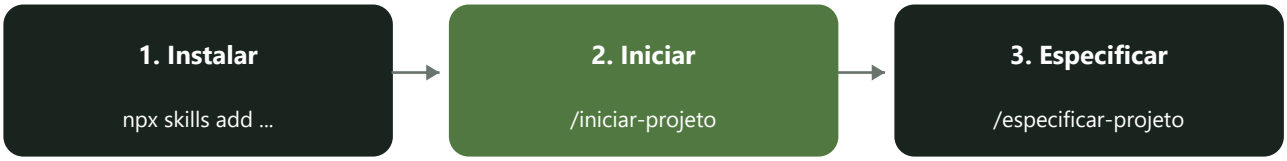
ESCOLHA RÁPIDA

Feature completa: implementar-sdd
Somente requisitos: especificar-feature
Somente testes: criar-testes
Somente revisão: revisar-codigo
Commits + versão + envio: versionar
Tag de uma versão pronta: publicar-release

3. COMO UTILIZAR AS SKILLS EM UM PROJETO REAL

A. Preparando o Projeto

EXECUTA SOMENTE UMA VEZ NO INÍCIO



1. Instalando Skills

Na raiz do projeto que utilizará as skills, execute o comando do README:

INSTALAÇÃO

```
npx skills add paulocesaaars/skills -a claude-code -a cursor -a github-copilot --copy -y
```

Versione **.claude/skills/**, **.agents/skills/** e **skills-lock.json**. Acrescente **-g** para instalação global, sem versionar por projeto; use **-s nome-da-skill** para selecionar uma. Atualize com **npx skills update -y** e confira as alterações.

2. Iniciando Projeto

Execute **/iniciar-projeto**. A skill cria, quando faltam, AGENTS.md, CLAUDE.md, HARNESS.md, PRD.md, TDD.md, DESIGN.md, BDD.md e bdd/. Preserva conteúdo existente e verifica ferramentas. Pastas da aplicação serão definidas no TDD; esta etapa prepara o contexto.

3. Especificando Projeto

Execute **/especificar-projeto**. Informe se o repositório é o projeto inteiro ou parte de um maior; nesse caso, forneça a especificação geral ou links para os outros repositórios.

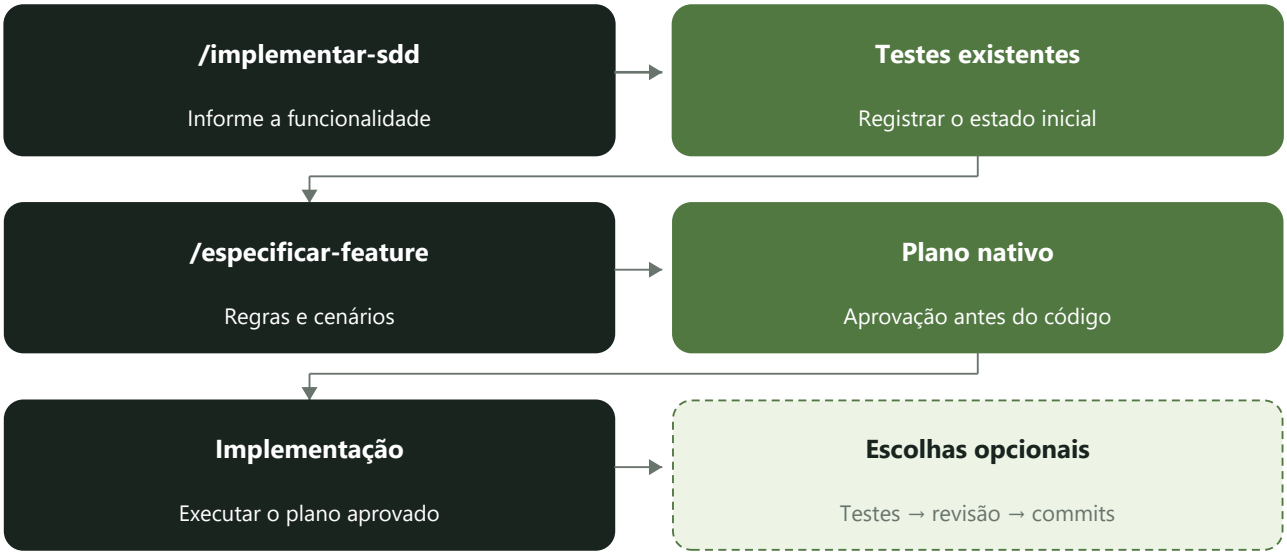


Primeira entrega: delimite objetivo, público, inclusões e exclusões. Exemplo: “Registrar, listar e encerrar chamados; sem anexos”. O repositório não define uma etapa chamada MVC. Se desejar Model-View-Controller, registre essa escolha no TDD. A implementação vem no ciclo B.

3 • B. ADICIONANDO UMA FEATURE

1. Especificando Feature

Você pode usar **implementar-sdd** para coordenar o fluxo completo, ou executar as skills individualmente. O esquema abaixo representa a sequência interna da skill unificada.



PEDIDO PARA A EXECUÇÃO INDIVIDUAL

Use especificar-feature para permitir que um atendente encerre um chamado aberto. Não inclua reabertura. Defina permissões, erros e resultado esperado.

Resultado que você deve conferir

A história descreve o valor: “Como atendente, quero encerrar um chamado para indicar a resolução”. Os cenários tornam o comportamento verificável. Abaixo, um exemplo ilustrativo, a ser validado para o seu produto:

EXEMPLO DE CENÁRIO

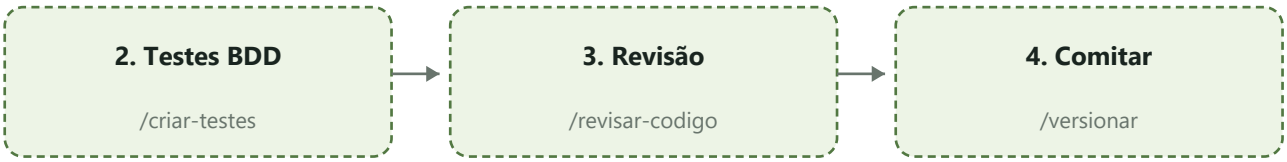
```
@US-02.1
Dado um chamado aberto e um atendente autorizado
Quando ele solicita o encerramento
Então o chamado fica com status encerrado
```

Se usar BDD, aprove a gravação dos cenários e mantenha o índice BDD.md coerente com bdd/. Se quiser salvar a especificação, autorize o caminho. Depois, ative o modo de planejamento nativo e aprove o plano.

No caminho manual: solicite a implementação do plano; não existe uma skill separada de implementação. **No unificado:** a skill faz essa transição. Se os testes iniciais falharem, você decide se a correção entra na especificação. Sem suíte, o fluxo avisa e segue.

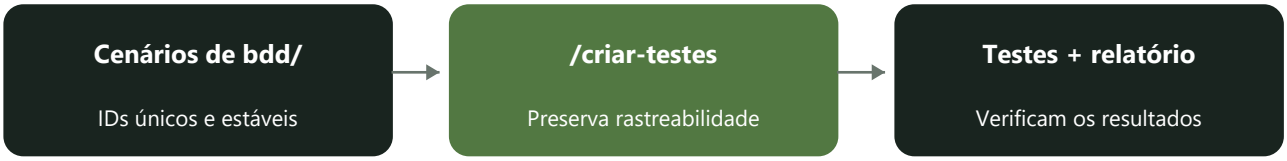
3 • B. ADICIONANDO UMA FEATURE

Testar, revisar e comitar



2. Gerar Teste do BDD (Opcional)

Peça **criar-testes** para um arquivo ou conjunto de cenários. BDD.md precisa estar preenchido e bdd/ deve conter cenários; arquivos vazios apenas mostram a estrutura. A skill identifica runner, marcação e testes faltantes e apresenta o plano antes de gravar.



Não há obrigação de relação 1:1 entre cenário e teste. A associação dos IDs deve permanecer clara. Cenários @manual ficam como checklist; testes existentes não são reescritos. Se o produto não cumprir o cenário, preserve a asserção e trate a correção separadamente.

3. Revisar Código (Opcional)

Peça **revisar-codigo** no diff e nos testes novos. A revisão verifica duplicação, SOLID, arquitetura, padrões e cobertura do comportamento. Entrega local, problema, impacto e sugestão; **não aplica refatorações**.

Resultado na implementar-sdd	Próximo passo
Crítico ou alto	Interromper. Você pede a correção, o agente aplica o que foi aceito e revisa novamente.
Médio ou baixo	Registrar e seguir para a escolha de versionamento.

4. Comitar (Opcional)

A skill atual é **versionar**. No fluxo completo, organiza commits, sobe a versão e envia a branch. Um pedido apenas de commit autoriza somente os commits; versão e push exigem autorização específica. Revise branch, arquivos e grupos antes de aprovar.

Testes ou checagens falhando bloqueiam commit e release. Sem suíte, as skills avisam que o funcionamento não foi validado. implementar-sdd termina sem iniciar documentação ou publicação e sem oferecer PR.

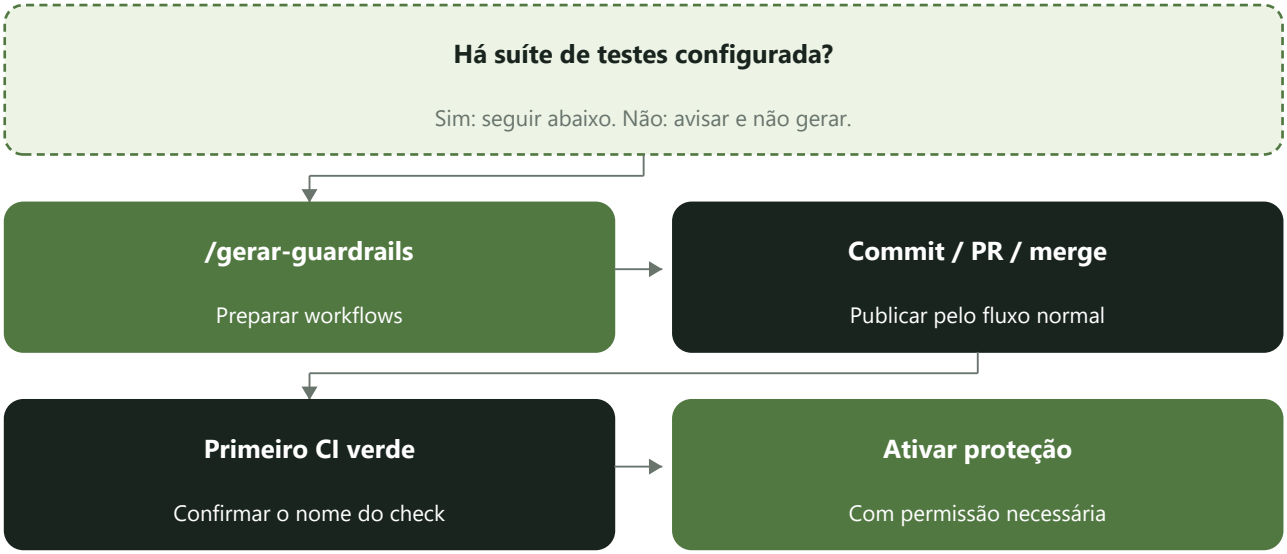
3 • C. DEPLOY E OPERAÇÃO

1. Gerar Guardrails (Opcional)

Uma vez e somente com testes configurados. Use gerar-guardrails para preparar CI de pull request, integrar testes à release e exigir o check verde na branch principal do GitHub.

PEDIDO

Use gerar-guardrails para este projeto com testes configurados. Reaproveite os workflows e apresente o plano antes de alterar arquivos ou regras.



Por que essa ordem?

Uma proteção que exige um check antes de ele existir pode deixar os PRs esperando indefinidamente. Faça o workflow entrar pelo fluxo normal e obtenha uma primeira execução verde antes de ativar a regra.

Situação encontrada	Como proceder
CI e proteção já completos	Não gerar novamente.
Sem permissão administrativa	Preparar workflows e entregar o pedido ao administrador; registrar a proteção como pendente.
Host diferente de GitHub	Receber o plano e as orientações, sem alterações remotas executadas pela skill.
Workflow gerado	Ainda precisa ser commitado e enviado: gerar-guardrails não faz commit nem push.

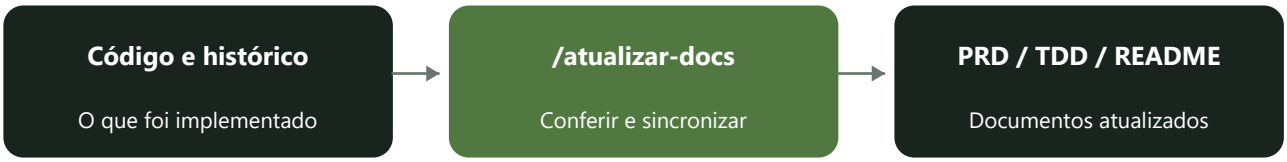
Confira a saída: sintaxe do workflow, execução dos comandos, check exigido e regra ativa ou pendência explícita. Um arquivo YAML sozinho não confirma proteção. Ter CI verde também não prova que testes foram coletados e executados.

3 • C. DEPLOY E OPERAÇÃO

Atualizar docs e fechar a entrega

2. Atualizar Docs

Use **/atualizar-docs**. No início, escolha: commitar, publicar release ou nenhum. A skill compara código e histórico, preserva edições locais e sincroniza PRD, TDD e README. Não altera os cenários de BDD; lacunas são relatadas.

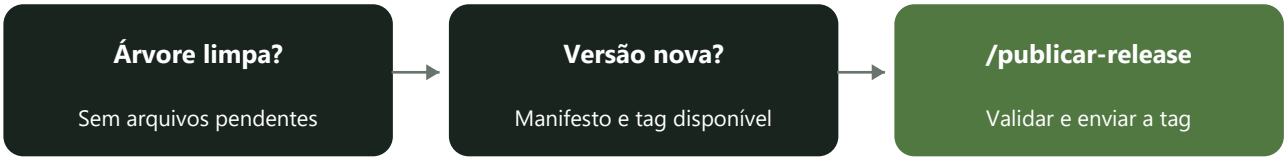


EXEMPLO DE PEDIDO

Use atualizar-docs e, ao final, publique a release. Primeiro registre a documentação pendente conforme o fluxo da skill.

3. Comitar ou Publicar Release

Escolha no início	Fechamento
Nenhum	Receber o relatório e encerrar sem commit ou release.
Commitar	Seguir versionar, respeitando o escopo autorizado de commit, versão e envio.
Publicar release	Commitar apenas os documentos alterados, sem subir a versão novamente, e então chamar publicar-release.



Antes de publicar: confira branch, testes, checagens, versão e notas. publicar-release não commita nem altera a versão. Outras mudanças pendentes precisam ser resolvidas antes; uma tag existente não é recriada.

Tag, release e deploy são diferentes. Tag é uma referência Git; release é uma versão publicada; deploy é a implantação em um ambiente. O pipeline determina se o envio da tag publica artefatos ou implanta algo. Sem pipeline, a tag não faz isso automaticamente.

Se houver pipeline, acompanhe o resultado e os artefatos. Para a próxima funcionalidade, volte ao ciclo B; não repita a preparação inicial nem gere guardrails já existentes.

Referência do conteúdo: README e catálogo paulocesaaars/skills, conferidos na cópia local. Nomes atuais: versionar (substitui commitar) e rastreabilidade BDD sem exigência de 1:1.